JustYours.

SMART CONTRACT AUDIT

ZOKYO.

Oct 25th, 2021 | v. 1.0

PASS

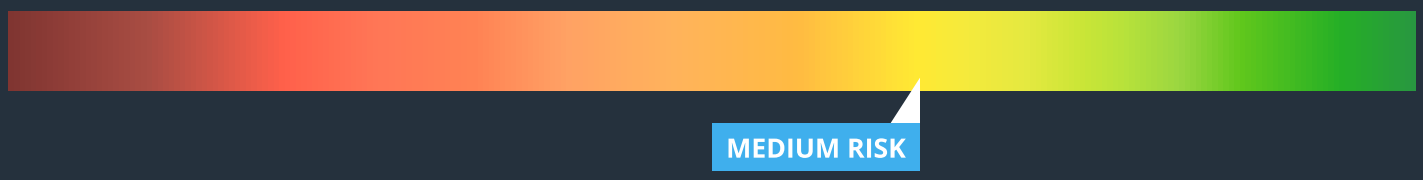
Zokyo Security team audited the given smart contract and provided the list of operational and security issues found.

TECHNICAL SUMMARY

This document outlines the overall security of the JustYours smart contracts, evaluated by Zokyo's Blockchain Security team.

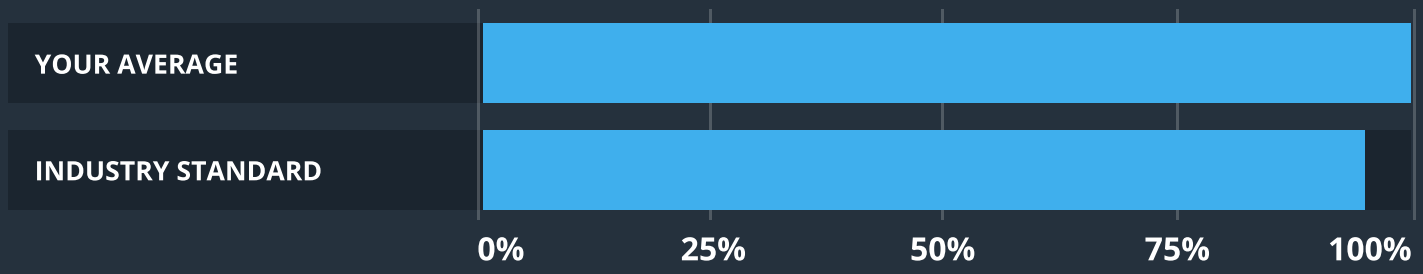
The scope of this audit was to analyze and document the JustYours smart contract codebase for quality, security, and correctness.

Contract Status



There were no critical issues found during the audit.

Testable Code



The testable code is 100%, which is above the industry standard of 95%.

It should be noted that this audit is not an endorsement of the reliability or effectiveness of the contract, rather limited to an assessment of the logic and implementation. In order to ensure a secure contract that's able to withstand the Ethereum network's fast-paced and rapidly changing environment, we at Zokyo recommend that the JustYours team put in place a bug bounty program to encourage further and active analysis of the smart contract.

TABLE OF CONTENTS

Auditing Strategy and Techniques Applied . . . 3
Summary . . . 4
Structure and Organization of Document . . . 5
Complete Analysis . . . 7
Code Coverage and Test Results for all files . . . 9
Tests written by Zokyo Secured team . . . 9

AUDITING STRATEGY AND TECHNIQUES APPLIED

The Smart contract's source code was taken from the JustYours repository.

Repository:
https://ftmscan.com/address/0x37c045be4641328dfeb625f1dde610d061613497

Functions under the scope:

- JUST Token

Throughout the review process, care was taken to ensure that the token contract:

- Implements and adheres to existing Token standards appropriately and effectively;
- Documentation and code comments match logic and behavior;
- Distributes tokens in a manner that matches calculations;
- Follows best practices in efficient use of gas, without unnecessary waste;
- Uses methods safe from reentrance attacks;
- Is not affected by the latest vulnerabilities;
- Whether the code meets best practices in code readability, etc.

Zokyo's Security Team has followed best practices and industry-standard techniques to verify the implementation of JustYours smart contracts. To do so, the code is reviewed line-by-line by our smart contract developers, documenting any issues as they are discovered. Part of this work includes writing a unit test suite using the Truffle testing framework. In summary, our strategies consist largely of manual collaboration between multiple team members at each stage of the review:

1	Due diligence in assessing the overall code quality of the codebase.	3	Testing contract logic against common and uncommon attack vectors.
2	Cross-comparison with other, similar smart contracts by industry leaders.	4	Thorough, manual review of the codebase, line-by-line.

SUMMARY

The Zokyo team has conducted a security audit of the given codebase. The contracts provided for an audit are well written and structured. All the findings within the auditing process are presented in this document.

There were no critical issues found during the auditing process. The security team has found 2 informational issues and 3 issues with high severity. None of them was resolved by the JustYours team as the contract has already been deployed on the mainnet.

As the issues are not fixed by the JustYours team and taken into consideration the severity level of the issues found, we can give a score of 82 to the provided codebase.

STRUCTURE AND ORGANIZATION OF DOCUMENT

For ease of navigation, sections are arranged from most critical to least critical. Issues are tagged “Resolved” or “Unresolved” depending on whether they have been fixed or addressed. Furthermore, the severity of each issue is written as assessed by the risk of exploitation or other unexpected or otherwise unsafe behavior:

Critical

The issue affects the ability of the contract to compile or operate in a significant way.

High

The issue affects the ability of the contract to compile or operate in a significant way.

Medium

The issue affects the ability of the contract to operate in a way that doesn’t significantly hinder its behavior.

Low

The issue has minimal impact on the contract’s ability to operate.

Informational

The issue has no impact on the contract’s ability to operate.

COMPLETE ANALYSIS

Burn functions should be removed

HIGH | UNRESOLVED

Since the white paper states that, there would never be any burning of tokens, we advise the burn functions be removed as it's not a required function of the core ERC20 token standard.

Change accessibility specifier of mint function

HIGH | UNRESOLVED

As stated in the white paper that there would only be a fixed amount of $JUST token, the minting function should have a private accessibility modifier, so that after using it in the constructor, no other contract or person can call it outside the contract.

Total supply allocation upon deployment address should not be an EOA

HIGH | UNRESOLVED

The address used for minting tokens of JUST token, we could not verify whether it's a smart contract address, an EOA, or a multisig wallet.

Recommendation:
We recommend you use a smart contract address that is being controlled by a DAO or a multisig contract.

Change public accessibility modifiers to external

INFORMATIONAL | UNRESOLVED

All functions with public accessibility modifiers can be improved with external accessibility modifiers.

Make state variables that do not change constants

INFORMATIONAL | UNRESOLVED

State variables such as name, decimals, and symbols, can be declared with the constant accessibility modifiers since they do not change throughout the lifetime of the smart contract.

	JUST Token
Re-entrancy	Pass
Access Management Hierarchy	Pass
Arithmetic Over/Under Flows	Pass
Unexpected Ether	Pass
Delegatecall	Pass
Default Public Visibility	Pass
Hidden Malicious Code	Pass
Entropy Illusion (Lack of Randomness)	Pass
External Contract Referencing	Pass
Short Address/ Parameter Attack	Pass
Unchecked CALL Return Values	Pass
Race Conditions / Front Running	Pass
General Denial Of Service (DOS)	Pass
Uninitialized Storage Pointers	Pass
Floating Points and Precision	Pass
Tx.Origin Authentication	Pass
Signatures Replay	Pass
Pool Asset Security (backdoors in the underlying ERC-20)	Pass

CODE COVERAGE AND TEST RESULTS FOR ALL FILES

Tests written by Zokyo Security team

As part of our work assisting JustYours in verifying the correctness of their contract code, our team was responsible for writing integration tests using the Truffle testing framework.

Tests were based on the functionality of the code, as well as a review of the JustYours contract requirements for details about issuance amounts and how the system handles these.

Code Coverage

The resulting code coverage (i.e., the ratio of tests-to-code) is as follows:

FILE	% STMTS	% BRANCH	% FUNCS	% LINES
contracts/	100.00	81.25	100.00	100.00
IERC20.sol	100.00	100.00	100.00	100.00
IERC20Metadat.sol	100.00	100.00	100.00	100.00
JustYours.sol	100.00	81.25	100.00	100.00
All files	**100.00**	**81.25**	**100.00**	**100.00**

Test Results

```
Contract: JustYours
  Initialization
    ✓ SUCCESS: Should return JustYours as name (100ms)
    ✓ SUCCESS: Should return 6969696969690000000 as totalSupply (66ms)
    ✓ SUCCESS: Should return JUST symbol (62ms)
    ✓ SUCCESS: Should return 0xb4899210d77c439159A6fEffBAe963A4Cf4672d7 balance
      as 6969696969690000000
  ERC20 Token Transfer, Approve, TransferFrom
```

```
    Transfer
      ✓ SUCCESS: Increases recipient balance by 10 and decrease minter balance by 10 (158ms)
      ✓ SUCCESS: returns true if transfer is successful
      ✓ SUCCESS: Emits Transfer event with correct from, to, value (56ms)
      ✓ FAIL: On transfer more than account balance (602ms)
      ✓ FAIL: On transfer to zero account (61ms)
    Approval
      ✓ SUCCESS: Approves amount for spender (100ms)
      ✓ SUCCESS: Emits Approved Event (56ms)
      ✓ SUCCESS: returns true if approval is successful
      ✓ FAIL: approve to zero address spender (53ms)
      TransferFrom
        ✓ SUCCESS: Transfers approved amount to spender (151ms)
        ✓ SUCCESS: returns true if transferFrom is successful (80ms)
        ✓ SUCCESS: Emits Transfer event (101ms)
        ✓ FAIL: Transfer more than available allowance (92ms)
      Increase Allowance
        ✓ SUCCESS: Increases allowance of spender (86ms)
        ✓ SUCCESS: returns true if increaseAllowance is successful (81ms)
        ✓ FAIL: Allowing spender to spend more than account balance (140ms)
        ✓ FAIL: approve to zero spender (39ms)
      Decrease Allowance
        ✓ SUCCESS: Decreases allowance of spender (152ms)
        ✓ SUCCESS: returns true if decreaseAllowance is successful (302ms)
        ✓ FAIL: Decreasing more than approved for spender (382ms)
      Decimals
        ✓ SUCCESS: Returns 10 as decimals
        ✓ FAIL: Returns decimals not 1

  26 passing (8s)
```

We are grateful to have been given the opportunity to work with the JustYours team.

The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

Zokyo's Security Team recommends that the JustYours team put in place a bug bounty program to encourage further analysis of the smart contract by third parties.

ZOKYO.